

Indentation in Python

 <https://github.com/learn-co-curriculum/python-p3-python-indentation>  <https://github.com/learn-co-curriculum/python-p3-python-indentation/issues/new>

Learning Goals

- Understand how the interpreter uses indentation to group blocks of code.

Key Vocab

- **Code Block:** a collection of code that is interpreted together. Python groups code blocks by indentation level.

Introduction

Indentation is a central concept of Python programming. New lines with indentations are required after certain statements. Without proper indentation, your code will be interrupted by an `IndentationError`.

Why Are Indentations Necessary?

Imagine you are preparing a complicated meal that you have never made before. There is an appetizer, a salad, a main course, and a dessert. You likely have an idea as to where certain things go- the steak isn't likely to be part of the dessert- but there are other important ingredients in there like butter, vinegar, salt, and so on. It is important that these ingredients are listed with their respective dishes so that you can make a coherent (and delicious!) meal.

Python code is broken into **blocks** to be interpreted as a continuation of one idea- these are like the recipes for each course of your meal. Maybe you need to prepare a dressing and sun-dry some tomatoes for your salad; these can be written as smaller code blocks inside of the block for a recipe as a whole.

A code block begins with a **colon** `:` followed by a newline and an indentation. All code inside of a block must be indented to the same level to be read by the interpreter. Consistent indentation also makes your code much easier to read (just like in JavaScript).

Where Are Indentations Necessary?

Indentations are required on each new line inside of a code block. Code blocks can begin with a number of keywords or statements, but here are some of the most common:

- `class` Definitions
- Function and Method Definitions (following the keyword `def`)
- `if` , `elif` , and `else`
- `try` , `except` , and `finally`
- `for` and `while`

A Foolish Consistency is the Hobgoblin of Little Minds

(<https://peps.python.org/pep-0008/#a-foolish-consistency-is-the-hobgoblin-of-little-minds>)

[Python Enhancement Proposal 8 \(PEP 8, for short\)](https://peps.python.org/pep-0008/)  (<https://peps.python.org/pep-0008/>) gives writing conventions for Python programmers. One of Guido van Rossum's key observations that he kept in mind while developing Python was that *code is read much more often than it is written*. This means that readers need to be able to understand your code, so we need to adhere to certain standards while coding.

Spaces, not Tabs

PEP 8 guidelines state that indentations in Python should consist of spaces rather than tabs. Tabs are visually interpreted differently in different environments, so they are not advisable in production code.

Four Spaces Each Time

Every indentation in Python code should consist of four spaces. This is long enough to distinguish between indentation levels but short enough to provide space for indentation levels in several nested code blocks.

Indent Statements that Continue past One Line

Some functions and methods take a lot of arguments. **A lot** of arguments. Take this one, for instance:

```
def take_twenty_arguments(arg1, arg2, arg3, arg4, arg5, arg6, arg7, arg8, arg9, arg10, arg11, arg12, arg13, arg14, arg15, arg16, arg17, arg18, arg19, arg20):  
    '''Takes 20 arguments and does nothing.'''  
    pass
```



Our function definition here takes up *158 characters*. PEP 8 says that lines should be composed of 79 or fewer characters, so we're way off here.

Python supports *implicit line joining* inside of parentheses and brackets, which means that we can add a new line anywhere in our list of arguments as long as it does not interrupt the name of an argument. Let's break this function definition down a bit more:

```
def take_twenty_arguments(arg1, arg2, arg3, arg4, arg5, arg6, arg7, arg8,  
    arg9, arg10, arg11, arg12, arg13, arg14, arg15, arg16, arg17, arg18,  
    arg19, arg20):  
    '''Takes 20 arguments and does nothing.'''  
    pass
```

This is much cleaner, but the casual reader might have some trouble telling where the arguments end and the code block begins. PEP 8 provides us two solid options for avoiding this ambiguity:

```
# Aligning arguments with the opening delimiter
def take_twenty_arguments(arg1, arg2, arg3, arg4, arg5, arg6, arg7, arg8,
                           arg9, arg10, arg11, arg12, arg13, arg14, arg15,
                           arg16, arg17, arg18, arg19, arg20):
    '''Takes 20 arguments and does nothing.'''
    pass
```

OR

```
# Hanging four-space indents with an added level
def take_twenty_arguments(
    arg1, arg2, arg3, arg4, arg5, arg6, arg7, arg8, arg9, arg10, arg11,
    arg12, arg13, arg14, arg15, arg16, arg17, arg18, arg19, arg20):
    '''Takes 20 arguments and does nothing.'''
    pass
```

These options are not restricted to statements that extend past 79 characters; you should take the extra effort to write readable code using the PEP 8 style guide whenever possible.

```
# Creating a dict
my_dict = {
    "word_1": "definition 1",
    "word_2": "definition 2",
}
```

```
# Creating a list
my_list = [
    "item_1",
    "item_2",
]
```

Resources

- [Python PEP 8 Style Guide](https://peps.python.org/pep-0008/)  [\(https://peps.python.org/pep-0008/\)](https://peps.python.org/pep-0008/)
- [How to Write Beautiful Python Code with PEP 8](https://realpython.com/python-pep8/)  [\(https://realpython.com/python-pep8/\)](https://realpython.com/python-pep8/)